

Ejercicio de decision

1. Descarga el archivo pacientes2 y determina cual de ellos presentará una posible enfermedad, de acuerdo con las variables que se encuentran en él.
2. Revisa las 5 primeras filas.
3. Revisa la información del set de datos.
4. Cuanta la cantidad de pacientes que pertenecen a la clase SI y NO.
5. Realiza un countplot de la variable enfermedad (investiga como hacerlo).
6. Realiza la descripción estadística.
7. Revisa los datos nulos.
8. Separa las variables X y Y, donde Y es la variable enfermedad.
9. Separa el set de datos en entrenamiento y prueba
10. Crea el modelo de árbol de decisión donde solo se vean 4 niveles.
11. Realiza el entrenamiento del modelo.
12. Muestra el gráfico del árbol de decisión
13. Realiza las predicciones, utilizando los datos de prueba
14. Realiza las métricas de medición del modelo: matriz de confusión y accuracy

CODIGO COMPLETO,

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Apr  6 22:20:45 2026
4
5  @author: bombo
6  """
7
8  # Importaciones
9  import pandas as pd
10 import numpy as np
11 import seaborn as sns
12 import matplotlib.pyplot as plt
13 from sklearn.model_selection import train_test_split
14 from sklearn.tree import DecisionTreeClassifier, plot_tree
15 from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
16
17 # Cargar datos
18 df = pd.read_csv("~/spyder-py3/pacientes2.csv")
19
20 # Exploración inicial
21 print(df.head())
22 print(df.info())
23 print(df['Enfermedad'].value_counts())
24
25 # Visualización
26 sns.countplot(data=df, x='Enfermedad')
27 plt.show()
28
29 # Estadísticas
30 print(df.describe())
31
32 # Nulos
33 print(df.isnull().sum())
34
35 # Separar X e Y
36 X = df.drop('Enfermedad', axis=1)
37 Y = df['Enfermedad']
38
39 # Dividir
40 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42, stratify=Y)
```

```

37 Y = df['Enfermedad']
38
39 # Dividir
40 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42, stratify=Y)
41
42 # Modelo
43 modelo = DecisionTreeClassifier(max_depth=4, random_state=42)
44 modelo.fit(X_train, Y_train)
45
46 # Visualizar árbol
47 plt.figure(figsize=(20,10))
48 plot_tree(modelo, feature_names=X.columns, class_names=['No', 'Si'], filled=True, rounded=True)
49 plt.show()
50
51 # Predicciones
52 Y_pred = modelo.predict(X_test)
53
54 # Métricas
55 print(confusion_matrix(Y_test, Y_pred))
56 print(f"Accuracy: {accuracy_score(Y_test, Y_pred):.4f}")
57 print(classification_report(Y_test, Y_pred, target_names=['No', 'Si']))

```

EJECUCION DE CODIGO.

```

In [10]: %runfile 'C:/Users/bombo/.spyder-py3/Sin título1.py' --wdir
NOEXPED Enfermedad HIPERTEN HIPERGLU ... GENERO FUMA ALCOHOL POLIURIA
0      1         NO         0         0 ...      0      0         0         0
1      2         SI         0         1 ...      1      0         0         0
2      3         SI         1         1 ...      1      0         0         1
3      4         NO         1         1 ...      0      0         0         0
4      5         NO         0         0 ...      0      0         0         0

[5 rows x 12 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 985 entries, 0 to 984
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   NOEXPED     985 non-null   int64
1   Enfermedad  985 non-null   object
2   HIPERTEN    985 non-null   object
3   HIPERGLU    985 non-null   object
4   GENERO      985 non-null   object
5   FUMA        985 non-null   object
6   ALCOHOL     985 non-null   object
7   POLIURIA    985 non-null   object

```

```
Terminal 1/A X
POLIURIA      0
dtype: int64
[[133  5]
 [ 13 46]]
Accuracy: 0.9086
      precision    recall  f1-score   support

     No       0.91      0.96      0.94       138
     Si       0.90      0.78      0.84        59

 accuracy          0.91          0.91       197
  macro avg       0.91      0.87      0.89       197
weighted avg       0.91      0.91      0.91       197

In [11]:
```

Terminal de IPython Historial



